

Complements of Operational Research

5.1 Dynamic programming: Bellman's principle and the shortest path

José Firmino Aguilar Madeira

Deciding in stages

Many problems are a **sequence of decisions**: which road to take, which product to load, how many people to have each week.

Enumerating every sequence grows far too fast.

Dynamic programming (DP) solves the problem **backwards**, one stage at a time, reusing what it has already computed.

Principle of optimality (Bellman, 1957)

An optimal policy has the property that, whatever the initial state and initial decision are, the remaining decisions must form an optimal policy with respect to the state resulting from the first decision.

Ingredients

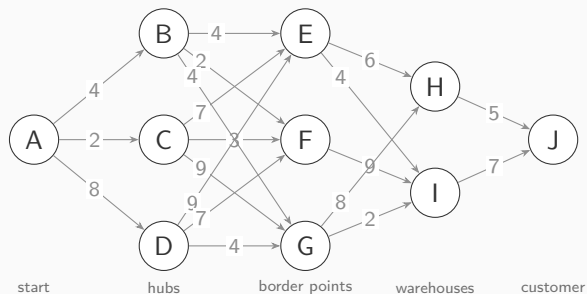
- **stages** $n = 1, 2, \dots, N$;
- **state** s : what we need to know to decide from here on;
- **decision** x at each stage;
- **recursion**, with new state $s' = T(s, x)$:

$$f_n(s) = \min_x \{c_n(s, x) + f_{n+1}(T(s, x))\}.$$

In the three problems of this chapter, the new state is the decision itself (the next node, the remaining capacity, this week's workers).

In this chapter: shortest path (5.1), knapsack (5.2) and workforce planning (5.3).

The example: taking a load from A to J



A haulier takes a load from A to the customer J, passing through a logistics hub (B, C, D), a border crossing point (E, F, G) and a distribution warehouse (H, I).

On the arcs: cost of each leg, in hundreds of €.

Question: what is the cheapest route?

There are 15 possible routes.

Always choose the cheapest leg? No.

Greedy rule (short-sighted): at each node, follow the cheapest arc.

- $A \rightarrow C$ (2), $C \rightarrow F$ (3), $F \rightarrow I$ (9), $I \rightarrow J$ (7).
- Total: $2 + 3 + 9 + 7 = 21$.

The cheap arc $A \rightarrow C$ leads to a part of the network where everything that follows is expensive. A decision can only be assessed **together with its consequences**.

The idea of DP

Start at the end. For each node, compute the minimum cost from there to J, $f(\text{node})$. When we reach A, we already know everything.

$$f(i) = \min_j \{c_{ij} + f(j)\}, \quad f(J) = 0.$$

Backward recursion

Stage 4 (warehouses): $f(H) = 5$, $f(I) = 7$.

Stage 3 (border points)

node	via H	via I	f	decision
E	$6 + 5 = 11$	$4 + 7 = 11$	11	H or I
F	—	$9 + 7 = 16$	16	I
G	$8 + 5 = 13$	$2 + 7 = 9$	9	I

Stage 1 (start)

node	via B	via C	via D	f	decision
A	$4 + 13 = 17$	$2 + 18 = 20$	$8 + 13 = 21$	17	B

Stage 2 (hubs)

node	via E	via F	via G	f	dec.
B	15	18	13	13	G
C	18	19	18	18	E or G
D	20	23	13	13	G

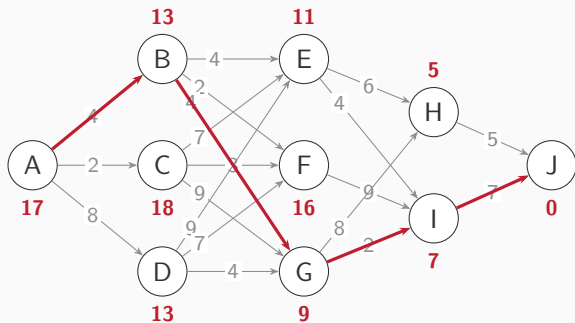
(for example, B via G: $4 + f(G) = 4 + 9 = 13$)

Reading the solution forwards

$A \rightarrow B$ (decision at A) $\rightarrow G$ (at B) $\rightarrow I$ (at G) $\rightarrow J$.

Minimum cost: **17** (1700 €), against 21 for the greedy rule.

The optimal route



in red: $f(\text{node})$, minimum cost to J

$A \rightarrow B \rightarrow G \rightarrow I \rightarrow J$, cost 17.

The f values give more than the answer: they give the best route **from any node**. If the load is already at C (for example, because of a detour), the best is to go via E or via G, at cost 18.

It is this “policy for every state” that distinguishes DP from a simple path search.

How much is saved?

In this example: enumerating the 15 routes takes $15 \times 3 = 45$ additions; DP does one addition per arc: 19.

In general: with N stages and k nodes per stage,

- enumeration: k^N routes;
- DP: about $N k^2$ additions.

N	k	routes	DP additions
4	3	81	36
10	10	10^{10}	1000
20	10	10^{20}	2000

For networks without stages

In general networks with non-negative costs (city streets, power grid), **Dijkstra's** algorithm is normally used. It shares the same idea of optimal substructure, although it is not the stage-by-stage recursion used here. It is behind navigation systems.

DP requires the problem to decompose into stages and the state to summarise everything that matters from the past (Markov property).

```
arcos = {'A': {'B': 4, 'C': 2, 'D': 8},
        'B': {'E': 4, 'F': 2, 'G': 4},
        'C': {'E': 7, 'F': 3, 'G': 9},
        'D': {'E': 9, 'F': 7, 'G': 4},
        'E': {'H': 6, 'I': 4}, 'F': {'I': 9},
        'G': {'H': 8, 'I': 2}, 'H': {'J': 5}, 'I': {'J': 7}}
ordem = ['H', 'I', 'E', 'F', 'G', 'B', 'C', 'D', 'A'] # from the end
f, dec = {'J': 0}, {}
for i in ordem:
    dec[i] = min(arcos[i], key=lambda j: arcos[i][j] + f[j])
    f[i] = arcos[i][dec[i]] + f[dec[i]]
no, caminho = 'A', ['A']
while no != 'J':
    no = dec[no]; caminho.append(no)
print(f['A'], caminho) # 17 ['A', 'B', 'G', 'I', 'J']
```

Each node is computed once, after its successors.

For large networks:

```
networkx.shortest_path(G, 'A', 'J',
weight='custo').
```

Full file (shortest path by stages):

```
code/python/05_dynamic_programming/
05_1_bellman_shortest_path/
caminho.py
```

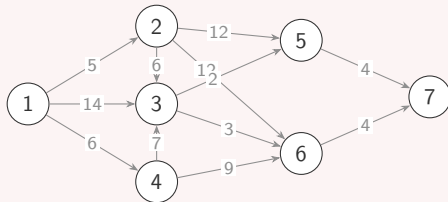
Example 1 — shortest path from node 1 to node 7

Distances: $d(1, 2) = 5$, $d(1, 3) = 9$, $d(1, 4) = 8$,
 $d(2, 5) = 10$, $d(2, 6) = 17$, $d(3, 5) = 4$, $d(3, 6) = 10$,
 $d(4, 5) = 9$, $d(4, 6) = 9$, $d(5, 7) = 19$, $d(6, 7) = 9$.

Example 2 — shortest path from node 1 to node 7

$d(1, 2) = 7$, $d(1, 3) = 8$, $d(1, 4) = 9$, $d(2, 5) = 12$,
 $d(3, 5) = 8$, $d(3, 6) = 9$, $d(4, 5) = 7$, $d(4, 6) = 13$,
 $d(5, 7) = 9$, $d(6, 7) = 6$.

Example 3 — shortest path from node 1 to node 7



In each case: define stages and states, carry out the backward recursion and give all the optimal routes.

KEY TAKEAWAYS — 5.1

1. DP: decompose into **stages**, define the **state**, write the **recursion** and solve from the end to the start.
2. Bellman's principle: from any state, the rest of an optimal policy is optimal.
3. Shortest path: $f(i) = \min_j \{c_{ij} + f(j)\}$, $f(\text{destination}) = 0$; read the solution forwards.
4. The greedy rule can fail (21 against 17). DP assesses each decision together with its consequences.
5. The f values give the best decision from *any* state.

Next: 5.2 The knapsack problem

References and resources

References Full list: *Course bibliography*.

- Hillier, F. S., Lieberman, G. J., *Introduction to Operations Research*, 11th ed., McGraw-Hill, 2021 — Dynamic Programming (the stagecoach problem).
- Bellman, R., *Dynamic Programming*, Princeton University Press, 1957.
- Kasana, H. S., Kumar, K. D., *Introductory Operations Research*, Springer, 2004 — ch. 12.
- Taha, H. A., *Operations Research: An Introduction*, 10th ed., Pearson, 2017 — deterministic dynamic programming.

Resources

- Exercise sheet 5. Practical assignment no. 2: create and solve 2 dynamic programming problems.